

# DataFrame · groupby · pivot\_table 비교

같은 데이터를 보는 세 가지 시선 | 광명코딩학원

> DataFrame은 데이터를 저장하는 기본 표, groupby는 한 가지 기준으로 묶어 세로로 집계한 것, pivot\_table은 두 가지 기준으로 교차시켜 가로x세로 표로 펼친 것입니다. 셋은 서로 대체재가 아니라, 같은 데이터를 다른 형태로 바라보는 단계입니다.

## 1. DataFrame — 원본 데이터 (긴 형식)

행 하나가 "하나의 사건/관측치"를 의미하는 기본 표. 행 번호(0, 1, 2...) 자체에는 분석적 의미가 없고, 그냥 데이터를 저장하기 좋은 형태입니다.

```
from pandas import DataFrame

data = {
    "학생": ["철수", "철수", "철수", "영희", "영희", "영희"],
    "과목": ["수학", "영어", "과학", "수학", "영어", "과학"],
    "점수": [85, 90, 78, 92, 88, 95]
}

df = DataFrame(data)
print(df)
#   학생  과목  점수
# 0  철수  수학   85
# 1  철수  영어   90
# 2  철수  과학   78
# 3  영희  수학   92
# 4  영희  영어   88
# 5  영희  과학   95
```

주의: 이 상태로는 "철수 영어 점수는?" 같은 질문에 답하려면 6행을 다 훑어봐야 합니다.

## 2. groupby — 한 가지 기준으로 묶어 집계 (세로로 쌓임)

특정 컬럼 값을 기준으로 행을 묶고, mean/sum 같은 집계함수를 적용합니다. 결과는 보통 Series(1차원) 또는 MultiIndex로 세로로 길게 쌓인 형태입니다.

```
print(df.groupby("학생")["점수"].mean())
# 학생
# 영희    91.666667
# 철수    84.333333
# Name: 점수, dtype: float64
```

```
# 두 컬럼 기준으로 묶으면 MultiIndex로 쌓임
print(df.groupby(["학생", "과목"])["점수"].mean())
# 학생 과목
# 영희 과학 95.0
# 수학 92.0
# 영어 88.0
# 철수 과학 78.0
# 수학 85.0
# 영어 90.0
```

groupby 결과는 "다음 계산(추가 연산, 다른 함수 적용)으로 이어가기"에 편한 형태입니다.

### 3. pivot\_table — 두 가지 기준을 교차시켜 표로 펼치기 (가로x세로)

groupby와 같은 정보를 담지만, 행과 열 둘 다 의미 있는 카테고리로 펼쳐서 사람이 한눈에 비교하도록 만든 매트릭스(교차표)입니다.

```
print(df.pivot_table(index="학생", columns="과목", values="점수"))
# 과목 과학 수학 영어
# 학생
# 영희 95.0 92.0 88.0
# 철수 78.0 85.0 90.0
```

index는 행 기준, columns는 열 기준, values는 교차 칸에 채울 값. "철수 영어 점수는?" → 철수 행과 영어 열이 만나는 칸을 보면 바로 90.

## 4. 비교 요약표

| 구분          | 의미                              | 형태                                     | 예시 코드                                                           | 결과 형태                    |
|-------------|---------------------------------|----------------------------------------|-----------------------------------------------------------------|--------------------------|
| DataFrame   | 데이터 저장용<br>기본 표,<br>행 번호는 의미 없음 | 2차원<br>(의미 없는 행번호<br>+ 컬럼들)            | DataFrame(data)                                                 | 6행 3열,<br>순번일 뿐          |
| groupby     | 한 가지 기준으로<br>그룹 묶어 집계           | 1차원(Series)<br>또는 MultiIndex로<br>세로 쌓임 | df.groupby("학생")<br><br>["점수"].mean()                           | 학생별 평균이<br><br>세로로 출력    |
| pivot_table | 두 가지 기준을<br>교차시켜 가로x세로<br>표로 펼침 | 2차원<br>(행과 열 둘 다<br>의미 있는 카테고리)        | df.pivot_table(<br>index="학생",<br>columns="과목",<br>values="점수") | 학생(행) x 과목(열)<br><br>교차표 |

## 5. 추가 예시 — 같은 데이터, 세 가지 시선

같은 매출 데이터를 DataFrame(원본) -> groupby(세로 집계) -> pivot\_table(가로 교차) 순서로 비교해서 흐름을 따라가 봅니다.

```
data2 = {
    "지점": ["강남", "강남", "홍대", "홍대", "잠실", "잠실"],
    "카테고리": ["음료", "디저트", "음료", "디저트", "음료", "디저트"],
    "매출": [50000, 30000, 42000, 28000, 61000, 35000]
}
df2 = DataFrame(data2)
```

```
# ① 원본 (긴 형식)
print(df2)
#   지점  카테고리  매출
# 0  강남    음료  50000
# 1  강남    디저트  30000
# ...
```

```
# ② groupby (지점 기준 평균, 세로로 쌓임)
print(df2.groupby("지점")["매출"].mean())
# 지점
# 강남    40000.0
# 잠실    48000.0
# 홍대    35000.0
```

```
# ③ pivot_table (지점x카테고리 교차표)
print(df2.pivot_table(index="지점", columns="카테고리", values="매출"))
# 카테고리    디저트    음료
# 지점
# 강남      30000.0  50000.0
# 잠실      35000.0  61000.0
# 홍대      28000.0  42000.0
```

groupby는 "강남 지점 전체 평균이 얼마인가"를 보여주고, pivot\_table은 "강남 지점에서 음료와 디저트 매출이 각각 얼마인가"를 한 칸씩 보여줍니다. 분석 목적이 다르면 적합한 도구도 다릅니다.

[ 완료 ] DataFrame, groupby, pivot\_table의 차이와 의미를 비교했습니다. 셋은 경쟁 관계가 아니라 데이터를 단계적으로 바라보는 서로 다른 시선이라는 점을 기억하세요.